

LESSON 6: STRINGS & COLLECTIONS IN PYTHON

Python Core Engineering Series — In-Depth Study Notes & Architecture Reference

1. STRINGS & COLLECTIONS ARCHITECTURAL OVERVIEW

In Python, dealing with text data and grouping multiple items into organized data structures are core tasks. Rather than treating collections uniformly, Python provides specialized data structures with distinct properties regarding mutability, ordering, indexing, and uniqueness.

The Four Primary Collections + Strings Matrix

Data Structure	Type Name	Syntax	Ordered?	Mutable?	Duplicates Allowed?	Indexing / Slicing?	Primary Use Case
String	str	"" or ""	Yes	No (Immutable)	Yes (Repeated chars)	Yes	Text representation & formatting
List	list	[]	Yes	Yes (Mutable)	Yes	Yes	Dynamic, ordered arrays of items
Tuple	tuple	()	Yes	No (Immutable)	Yes	Yes	Read-only records, fixed coordinates
Set	set	{}	No	Yes (Mutable)	No (Strictly unique)	No (Hash-based)	Mathematical sets, duplicate elimination

Data Structure	Type Name	Syntax	Ordered?	Mutable?	Duplicates Allowed?	Indexing / Slicing?	Primary Use Case
Dictionary	dict	{k: v}	Yes (3.7+)	Yes (Mutable)	Keys: No / Values: Yes	Via Key (d[k])	Fast key-value lookups, JSON data

2. DEEP DIVE: STRINGS (str) & STRING IMMUTABILITY

A Python string is an immutable sequence of Unicode characters.

2.1 Immutability Mechanics

Immutability means that once allocated in memory, individual characters cannot be overwritten or reassigned:

Python

```
text = "Python"
```

```
# text[0] = "J" # ❌ TypeError: 'str' object does not support item assignment
```

Every string operation (`.replace()`, `.upper()`, `.strip()`) returns a brand-new string object in memory. The original string remains completely unchanged unless the variable name is explicitly rebound to the new object.

2.2 Core String Methods

Method Category	Method Syntax	Description	Example Input → Output
Case Alteration	<code>.upper()</code> / <code>.lower()</code>	Converts all characters to uppercase / lowercase	"Py" -> "PY" / "py"

Method Category	Method Syntax	Description	Example Input → Output
Case Formatting	<code>.title()</code> / <code>.capitalize()</code>	Capitalizes each word / Capitalizes first character only	"hello world" -> "Hello World"
Trimming	<code>.strip()</code> , <code>.lstrip()</code> , <code>.rstrip()</code>	Removes leading/trailing whitespace or target chars	" hi ".strip() -> "hi"
Search & Count	<code>.find(sub)</code>	Returns lowest index of substring (returns -1 if not found)	"code".find("d") -> 2
Search & Count	<code>.count(sub)</code>	Returns count of non-overlapping occurrences	"banana".count("a") -> 3
Substitution	<code>.replace(old, new)</code>	Replaces occurrences of old with new	"Java".replace("J", "N") -> "Nava"
Tokenization	<code>.split(sep)</code>	Splits string into a list based on delimiter (default: whitespace)	"a b c".split() -> ['a', 'b', 'c']
Reconstruction	<code>sep.join(iterable)</code>	Concatenates iterable of strings using caller as separator	"-".join(['a', 'b']) -> "a-b"

Method Category	Method Syntax	Description	Example Input → Output
Validation	<code>.isdigit()</code> / <code>.isalpha()</code>	Validates if string contains exclusively numbers / alphabets	<code>"123".isdigit()</code> -> True

3. SEQUENCE SLICING & NEGATIVE INDEXING

Strings, lists, and tuples share Python's unified sequence slicing interface.

3.1 Indexing Model

Python indices are zero-based from the left (0 to `len - 1`) and negative-based from the right (`-1` down to `-len`):

String: P y t h o n

Positive: 0 1 2 3 4 5

Negative: -6 -5 -4 -3 -2 -1

3.2 Slice Syntax

`sequence[start:stop:step]`

- **start:** Index where slice begins (inclusive, defaults to 0).
- **stop:** Index where slice ends (exclusive, stops at `stop - 1`).
- **step:** Stride increment (optional, defaults to 1).

Python

```
sample = "Python"
```

```
print(sample[0:3]) # 'Pyt' (indices 0, 1, 2)
```

```
print(sample[2:]) # 'thon' (index 2 to end)
```

```
print(sample[:4]) # 'Pyth' (start to index 3)
```

```
print(sample[::2]) # 'Pto' (every 2nd character)
```

```
print(sample[::-1]) # 'nohtyP' (string reversal via negative step)
```

4. LISTS (list): DYNAMIC MUTABLE ARRAYS

Lists are ordered, mutable collections capable of storing heterogeneous data types. In CPython, lists are implemented as dynamically resized arrays of pointers to heap objects.

4.1 Primary List Methods

Python

```
fruits = ["apple", "banana"]
```

Additions

```
fruits.append("mango")    # Adds to the very end -> ['apple', 'banana', 'mango']
```

```
fruits.insert(1, "grape") # Inserts at index 1 -> ['apple', 'grape', 'banana', 'mango']
```

```
fruits.extend(["kiwi", "fig"]) # Unpacks and appends multiple items
```

Removals

```
fruits.remove("banana")   # Removes first occurrence of value 'banana'
```

```
popped = fruits.pop()     # Removes & returns last item (or at index: fruits.pop(0))
```

```
del fruits[0]             # Deletes item at index 0 via keyword
```

Reordering (In-Place)

```
fruits.sort()             # Sorts list alphabetically/numerically in-place
```

```
fruits.reverse()          # Reverses list in-place
```

4.2 List Comprehensions

List comprehensions provide a clean, declarative syntax to construct lists from iterables in a single line, running faster than traditional `.append()` loops:

Syntax: `[expression for item in iterable if condition]`

Python

```
# Create a list of squares for even numbers only
```

```
even_squares = [x**2 for x in range(10) if x % 2 == 0]
```

Result: [0, 4, 16, 36, 64]

5. TUPLES (tuple): IMMUTABLE FIXED RECORDS

Tuples are ordered sequences of objects enclosed in parentheses (). Like strings, they are immutable.

5.1 Why Use Tuples Instead of Lists?

- **Data Integrity (Write-Protection):** Protects values that must not change during execution (e.g., GPS coordinates, RGB color constants, server port configs).
- **Performance:** Tuples consume less memory than lists and instantiate faster because their memory is allocated in a single fixed block.
- **Dictionary Keys:** Because tuples are immutable, they are hashable and can be used as dictionary keys (lists cannot).

5.2 Tuple Packing & Sequence Unpacking

Python

Packing

coordinates = (19.0760, 72.8777)

Unpacking into individual identifiers

lat, lon = coordinates

print(f"Latitude: {lat}, Longitude: {lon}")

The Single-Element Tuple Quirk: Must include trailing comma!

not_a_tuple = (5) # Type: int (evaluated as parentheses in math)

valid_tuple = (5,) # Type: tuple

6. SETS (set): HASH-BASED UNIQUE COLLECTIONS

A set is an unordered, mutable collection of unique, hashable objects defined with curly braces {}.

6.1 Key Set Characteristics

- **Automatic De-duplication:** Duplicate elements added to a set are silently ignored.
- **No Indexing:** Sets do not have an order; you cannot access items via `s[0]`.
- **$O(1)$ Average Lookup Complexity:** Membership testing (`item in my_set`) operates in near-instant $O(1)$ constant time using hash tables (unlike lists, which require $O(N)$ linear scans).

6.2 Mathematical Set Operations

Python

`A = {1, 2, 3, 4}`

`B = {3, 4, 5, 6}`

Union (All unique elements from both): `A | B`

`print(A | B)` # {1, 2, 3, 4, 5, 6}

Intersection (Common elements): `A & B`

`print(A & B)` # {3, 4}

Difference (Elements in A but not in B): `A - B`

`print(A - B)` # {1, 2}

Symmetric Difference (In A or B, but NOT both): `A ^ B`

`print(A ^ B)` # {1, 2, 5, 6}

7. DICTIONARIES (dict): KEY-VALUE HASH MAPS

Dictionaries map immutable, unique keys to arbitrary values. Since Python 3.7+, dictionaries preserve insertion order.

7.1 Safe Access: `dict[key]` vs `dict.get()`

- `dict["missing_key"]` raises a fatal `KeyError` if the key does not exist.

- `dict.get("missing_key", default)` safely returns `None` or a custom fallback value without throwing an exception.

Python

```
user = {"id": 101, "name": "Rahul"}
```

```
# ❌ Risky: Raises KeyError if 'role' is absent
```

```
# role = user["role"]
```

```
# ✅ Safe: Returns fallback "Guest"
```

```
role = user.get("role", "Guest")
```

7.2 Core Dictionary Operations

Python

```
student = {"name": "Priya", "grade": "A"}
```

```
# Adding / Updating
```

```
student["grade"] = "A+"    # Updates existing key
```

```
student["city"] = "Mumbai" # Adds brand new key-value pair
```

```
student.update({"age": 22, "status": "Active"})
```

```
# Removal
```

```
popped_val = student.pop("city") # Removes key and returns its value
```

```
del student["status"]    # Deletes key directly
```

```
# Iteration Paradigms
```

```
for k in student.keys(): ...    # Iterate over keys
```

```
for v in student.values(): ...  # Iterate over values
```



```
for k, v in student.items(): ... # Iterate over (key, value) tuples
```

8. COMPLETE SCRIPT EXAMPLE

Below is a single, self-contained Python script demonstrating string processing, collection manipulation, set math, dictionary transformations, and indexing:

Python

```
"""
```

Lesson 6: Strings & Collections Architecture

A complete demonstration of strings, lists, tuples, sets, dictionaries, slicing, list comprehensions, and practical data-processing pipelines.

```
"""
```

```
# =====
```

1. STRING TRANSFORMATIONS & METHOD CHAINING

```
# =====
```

```
print("--- 1. String Manipulation & Validation ---")
```

```
raw_header = " enterprise python data processing pipeline "
```

Method Chaining

```
clean_slug = raw_header.strip().title().replace(" ", "_")
```

```
print(f"Raw Input: '{raw_header}'")
```

```
print(f"Clean Slug: '{clean_slug}'")
```

Split and Join pipelines

```
log_entry = "2026-09-04|AUTH_SERVICE|LOGIN_SUCCESS|USER_992"
```

```
tokens = log_entry.split("|")
```

```
print(f"Tokenized Log: {tokens}")
```

```
reconstructed_log = " -> ".join(tokens)
print(f"Reconstructed: {reconstructed_log}")
```

```
# =====
```

2. LIST OPERATIONS & LIST COMPREHENSIONS

```
# =====
```

```
print("\n--- 2. List Mutations & Comprehensions ---")
```

```
data_points = [45, 12, 89, 33, 27, 89]
```

```
data_points.append(100)
```

```
data_points.insert(2, 50)
```

```
data_points.remove(89) # Removes only first 89
```

```
print(f"Mutated List: {data_points}")
```

```
# List Comprehension: Filter and transform in a single pass
```

```
filtered_scores = [x * 1.1 for x in data_points if x > 30]
```

```
print(f"Adjusted Scores (>30 with +10%): {[round(s, 1) for s in filtered_scores]}")
```

```
# =====
```

3. TUPLES: PACKING, UNPACKING & IMMUTABILITY

```
# =====
```

```
print("\n--- 3. Tuples & Coordinate Immutability ---")
```

```
server_endpoint = ("192.168.1.100", 8080, "TCP")
```

```
# Destructuring / Unpacking
```

```
ip_addr, port, protocol = server_endpoint
```

```
print(f"Bound Endpoint: Host={ip_addr} | Port={port} | Proto={protocol}")
```

```
# =====
```

4. SETS: DE-DUPLICATION & HASH MATHEMATICS

```
# =====
```

```
print("\n--- 4. Sets & Unique Operations ---")
```

```
customer_ids_batch_1 = [101, 102, 103, 104, 101, 102]
```

```
customer_ids_batch_2 = [103, 104, 105, 106]
```

```
set_a = set(customer_ids_batch_1)
```

```
set_b = set(customer_ids_batch_2)
```

```
print(f"Batch 1 Unique: {set_a}")
```

```
print(f"Batch 2 Unique: {set_b}")
```

```
print(f"Common Customers (Intersection): {set_a & set_b}")
```

```
print(f"All Unique Combined (Union): {set_a | set_b}")
```

```
print(f"New to Batch 2 Only (Diff B - A): {set_b - set_a}")
```

```
# =====
```

5. DICTIONARIES: KEY-VALUE AGGREGATIONS

```
# =====
```

```
print("\n--- 5. Dictionaries & Frequency Aggregation ---")
```

```
transactions = [
```

```
    {"user": "Alice", "amount": 250},
```

```
    {"user": "Bob", "amount": 120},
```

```
    {"user": "Alice", "amount": 300},
```

```
{ "user": "Charlie", "amount": 400},  
{ "user": "Bob", "amount": 80},  
]
```

Practical Pattern: Grouping / Aggregating totals by user

```
spending_totals = {}
```

```
for tx in transactions:
```

```
    user = tx["user"]
```

```
    amount = tx["amount"]
```

```
    # Safely get current accumulated total or start at 0
```

```
    spending_totals[user] = spending_totals.get(user, 0) + amount
```

```
print("Customer Aggregate Spending Ledger:")
```

```
for customer, total in spending_totals.items():
```

```
    print(f" User: {customer:<8} | Total Spent: ${total}")
```

```
# =====
```

6. UNIFIED SEQUENCE SLICING

```
# =====
```

```
print("\n--- 6. Universal Slicing ---")
```

```
alphabet_seq = "ABCDEFGHJIJ"
```

```
num_seq = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
```

```
print(f"String Substring [2:7]:   '{alphabet_seq[2:7]}'")
```

```
print(f"List Range [3:8]:         {num_seq[3:8]}")
```

```
print(f"Reversed Sequence [::-1]: '{alphabet_seq[::-1]}'")
```

```
print(f"Every 3rd Element [::3]: {num_seq[::3]}")
```

```
print(f"Last 3 Elements [-3:]: {num_seq[-3:]}")
```

9. CODE ARCHITECTURE BREAKDOWN: STEP-BY-STEP FLOW

1. **Tokenization Pipeline (split & join):** The raw log string is split using the pipe delimiter |. The resulting list of tokens is recombined into a formatted audit string using ->.
2. **Dynamic List Mutations & Filtering:** Items are modified in-place using .append(), .insert(), and .remove(). A list comprehension then filters items greater than 30 and applies a 10% multiplier in a single, clean step.
3. **Set Algebra for Fast Lookup:** Converting lists of customer IDs to sets eliminates duplicate records instantly. Using the & (intersection) and - (difference) operators executes fast set comparisons in memory without manual loops.
4. **Frequency & Spending Aggregation:** The dictionary pattern spending_totals.get(user, 0) + amount provides a clean way to aggregate incoming data without checking if the key exists first.

10. JUNIOR ANTI-PATTERNS & PERFORMANCE PITFALLS

Anti-Pattern 1: Inefficient String Concatenation in Loops

Python

❌ ANTI-PATTERN: Creates multiple temporary string objects in memory!

```
words = ["Python", "Is", "Fast"]
```

```
sentence = ""
```

```
for w in words:
```

```
    sentence += w + " " # Reallocates memory on every iteration: O(N^2)
```

✅ CORRECT: Single memory allocation using join

```
sentence = " ".join(words) # Fast: O(N)
```

Anti-Pattern 2: Using Lists for Frequent Membership Checks

Python

❌ SLOW: Linear scan O(N) lookup time on large collections

```
valid_ids = [1001, 1002, 1003, ..., 999999]
```

```
if user_id in valid_ids: # Must inspect elements sequentially  
    pass
```

```
# ✅ OPTIMAL: Set lookup runs in near-instant O(1) time
```

```
valid_ids_set = {1001, 1002, 1003, ..., 999999}
```

```
if user_id in valid_ids_set: # Instant hash-table lookup  
    pass
```

Anti-Pattern 3: Using a Mutable Object as a Default Parameter

Python

```
# ❌ DANGEROUS BUG: Default list is shared across all function calls!
```

```
def add_item(item, target_list=[]):
```

```
    target_list.append(item)
```

```
    return target_list
```

```
# ✅ CORRECT: Use None as default sentinel value
```

```
def add_item(item, target_list=None):
```

```
    if target_list is None:
```

```
        target_list = []
```

```
    target_list.append(item)
```

```
    return target_list
```

11. TECHNICAL INTERVIEW & RECAP CHEAT SHEET

High-Yield Questions & Answers

Q1: What is the key difference between a List and a Tuple in Python?

Answer: Lists are mutable (elements can be added, updated, or deleted in place) and are stored in dynamically allocated arrays. Tuples are immutable (cannot be changed after

creation) and have a smaller, fixed memory footprint. Because tuples are immutable, they are hashable and can be used as dictionary keys, whereas lists cannot.

Q2: What are the requirements for an object to be used as a Dictionary key or added to a Set?

Answer: The object must be hashable. An object is hashable if it has an immutable hash value that remains constant throughout its lifecycle (implements `__hash__()`) and can be compared for equality (implements `__eq__()`). Mutable objects like lists, sets, and dictionaries are unhashable and cannot serve as dictionary keys.

Q3: How do `dict.get(key)` and `dict[key]` differ when a key is missing?

Answer: Direct bracket access (`dict[key]`) raises a `KeyError` if the key is missing. The `.get(key, default)` method returns a fallback value (defaulting to `None`) instead of raising an exception, making it safer for dynamic lookups.

Q4: What is the time complexity of checking membership (`x in collection`) for a List versus a Set?

Answer: Checking membership in a list takes $O(N)$ linear time, as Python must scan elements sequentially. In a set, checking membership takes $O(1)$ constant time on average because sets use hash tables to jump directly to the memory bucket where the item resides.